Lab Report No. 01

Experiment Name

Implementation of DDA and Bresenham Line Drawing Algorithms to Draw an 'X' Shape Using OpenGL.

Objective

To implement both DDA (Digital Differential Analyzer) and Bresenham Line Drawing algorithms in a single OpenGL program and draw two lines that form an **'X' shape**.

Theory/Introduction

Line drawing is one of the fundamental operations in Computer Graphics. Different algorithms are used to determine the pixels required to display a straight line on the screen.

DDA (Digital Differential Analyzer) uses floating-point calculations to generate line points.

Bresenham Line Drawing Algorithm uses integer calculations, making it faster and more efficient.

In this experiment, the DDA algorithm is used to draw one line and the Bresenham algorithm is used to draw another line. The two lines intersect to form an **'X' shape**.

Algorithm/Procedure

DDA Algorithm

1. Input the starting and ending coordinates.
2. Calculate Δx and Δy.
3. Determine the number of steps using:
4. Steps = max(|Δx|, |Δy|)
5. Calculate x-increment and y-increment.
6. Plot the starting point.
7. Generate and plot the remaining points until the endpoint is reached.

Bresenham Algorithm

1. Input the starting and ending coordinates.
2. Calculate Δx and Δy.
3. Compute the initial decision parameter.
4. Plot the starting point.

5. Update the decision parameter at each step.
6. Select the next pixel accordingly.
7. Continue until the endpoint is reached.

Drawing the X Shape

1. Draw a line from (100,100) to (300,300) using DDA.
2. Draw a line from (300,100) to (100,300) using Bresenham.
3. Display both lines in the OpenGL window.

Source Code

```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *

def draw_pixel(x, y):
    glVertex2i(int(x), int(y))

# DDA Line Drawing Algorithm
def DDA(x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1

    steps = max(abs(dx), abs(dy))

    x_inc = dx / steps
    y_inc = dy / steps

    x = x1
    y = y1

    for i in range(steps + 1):
        draw_pixel(round(x), round(y))
        x += x_inc
        y += y_inc

# Bresenham Line Drawing Algorithm
def Bresenham(x1, y1, x2, y2):
    dx = abs(x2 - x1)
    dy = abs(y2 - y1)

    sx = 1 if x1 < x2 else -1
    sy = 1 if y1 < y2 else -1

    err = dx - dy

    while True:
        draw_pixel(x1, y1)
```

```
        if x1 == x2 and y1 == y2:
            break

        e2 = 2 * err

        if e2 > -dy:
            err -= dy
            x1 += sx

        if e2 < dx:
            err += dx
            y1 += sy

def display():
    glClear(GL_COLOR_BUFFER_BIT)

    glColor3f(1.0, 1.0, 1.0)

    glBegin(GL_POINTS)

    # First Line using DDA
    DDA(100, 100, 300, 300)

    # Second Line using Bresenham
    Bresenham(300, 100, 100, 300)

    glEnd()

    glutSwapBuffers()

glutInit()
glutInitDisplayMode(GLUT_RGBA | GLUT_DOUBLE)
glutInitWindowSize(500, 500)

glutCreateWindow(b"Marajul Islam - 28")

glClearColor(0.0, 0.0, 0.0, 1.0)

gluOrtho2D(0, 500, 0, 500)

glutDisplayFunc(display)
glutMainLoop()
```

Sample Input

Line 1: (100,100) → (300,300)

Line 2: (300,100) → (100,300)

Sample Output

An OpenGL window is displayed showing two intersecting diagonal lines that form an **'X' shape**.

Result

The DDA and Bresenham Line Drawing algorithms were successfully implemented. Two diagonal lines were drawn correctly, forming an **'X' shape** in the OpenGL window.

Conclusion

The experiment was completed successfully. Both DDA and Bresenham algorithms were used to draw straight lines. The output verified that the algorithms generated the required pixels correctly and produced the desired **'X' shape**.